

Regression Tree

Fitting Regression Trees

Here we fit a regression tree to the Boston data set. First, we create a training set, and fit the tree to the training data.

```
Boston <- read.csv("/cloud/project/Data/Boston.csv")
head(Boston)
```

```
##   X    crim zn indus chas   nox    rm  age    dis rad tax ptratio lstat medv
## 1 1 0.00632 18  2.31    0 0.538 6.575 65.2 4.0900   1 296    15.3  4.98 24.0
## 2 2 0.02731  0  7.07    0 0.469 6.421 78.9 4.9671   2 242    17.8  9.14 21.6
## 3 3 0.02729  0  7.07    0 0.469 7.185 61.1 4.9671   2 242    17.8  4.03 34.7
## 4 4 0.03237  0  2.18    0 0.458 6.998 45.8 6.0622   3 222    18.7  2.94 33.4
## 5 5 0.06905  0  2.18    0 0.458 7.147 54.2 6.0622   3 222    18.7  5.33 36.2
## 6 6 0.02985  0  2.18    0 0.458 6.430 58.7 6.0622   3 222    18.7  5.21 28.7
```

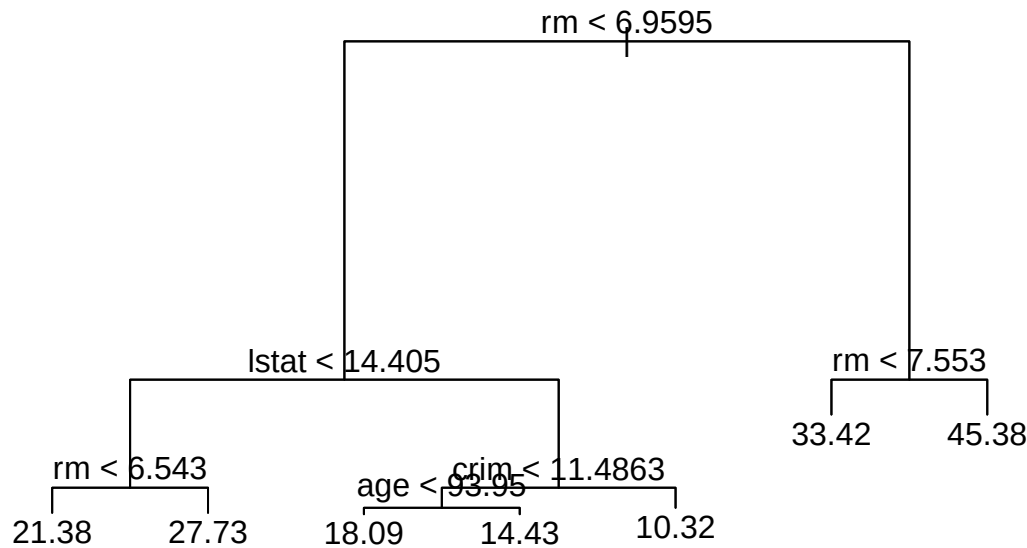
#there is an index variable named X, dropping it from the data set

```
Boston <- Boston[-1]
library(tree)
set.seed(1)
#50 to 50 split
train <- sample(1:nrow(Boston), nrow(Boston) / 2)
tree.boston <- tree(medv ~ ., Boston, subset = train)
summary(tree.boston)
```

```
##
## Regression tree:
## tree(formula = medv ~ ., data = Boston, subset = train)
## Variables actually used in tree construction:
## [1] "rm"    "lstat" "crim"  "age"
## Number of terminal nodes:  7
## Residual mean deviance:  10.38 = 2555 / 246
## Distribution of residuals:
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## -10.1800  -1.7770  -0.1775   0.0000   1.9230  16.5800
```

Notice that the output of `summary()` indicates that only four of the variables have been used in constructing the tree. In the context of a regression tree, the deviance is simply the sum of squared errors for the tree. We now plot the tree.

```
plot(tree.boston)
text(tree.boston, pretty = 0)
```



The variable `lstat` measures the percentage of individuals with {lower socioeconomic status}, while the variable `rm` corresponds to the average number of rooms. The tree indicates that larger values of `rm`, or lower values of `lstat`, correspond to more expensive houses. For example, the tree predicts a median house price of \$45,400 for homes in census tracts in which `rm` $\geq$ 7.553.

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
#performance of the model on the TrainData
```

```
#get the predicted values
```

```
PredictedTrain<- predict(tree.boston, newdata = Boston[train, ])
```

```
#create a data frame with 2 columns actual and predicted values
```

```
ModelTrain<-data.frame(obs = Boston[train, ]$medv, pred=PredictedTrain)
```

```
#calculate performance statistics
```

```
tree1<-defaultSummary(ModelTrain)
```

```
#lets compare this against regression model
```

```
f<-lm(medv~.,data=Boston[train,])
```

```
summary(f)
```

```
##
```

```
## Call:
```

```
## lm(formula = medv ~ ., data = Boston[train, ])
```

```
##
```

```
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -9.509 -2.614 -0.404   1.708  17.208
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  38.227423   6.575411   5.814 1.94e-08 ***
## crim        -0.106305   0.041564  -2.558  0.0112 *
## zn           0.029376   0.018408   1.596  0.1118
## indus        0.033199   0.076295   0.435  0.6639
## chas         2.449003   1.144370   2.140  0.0334 *
```

```
## nox          -13.505550   5.120878  -2.637   0.0089 **
## rm           4.003923   0.538257   7.439  1.80e-12 ***
## age          -0.001913   0.017164  -0.111   0.9113
## dis          -1.247533   0.271595  -4.593  7.05e-06 ***
## rad           0.366872   0.086694   4.232  3.30e-05 ***
## tax          -0.019621   0.004767  -4.116  5.30e-05 ***
## ptratio      -0.985512   0.176000  -5.599  5.86e-08 ***
## lstat        -0.507784   0.068368  -7.427  1.93e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.368 on 240 degrees of freedom
## Multiple R-squared:  0.7646, Adjusted R-squared:  0.7528
## F-statistic: 64.95 on 12 and 240 DF,  p-value: < 2.2e-16

PredictedTrain.reg<- predict(f, newdata = Boston[train, ])
#create a data frame with 2 columns actual and predicted values
ModelTrain.reg<-data.frame(obs = Boston[train, ]$medv, pred=PredictedTrain.reg)
#calculate performance statistics
reg1<-defaultSummary(ModelTrain.reg)
out<-rbind(tree1,reg1)
out

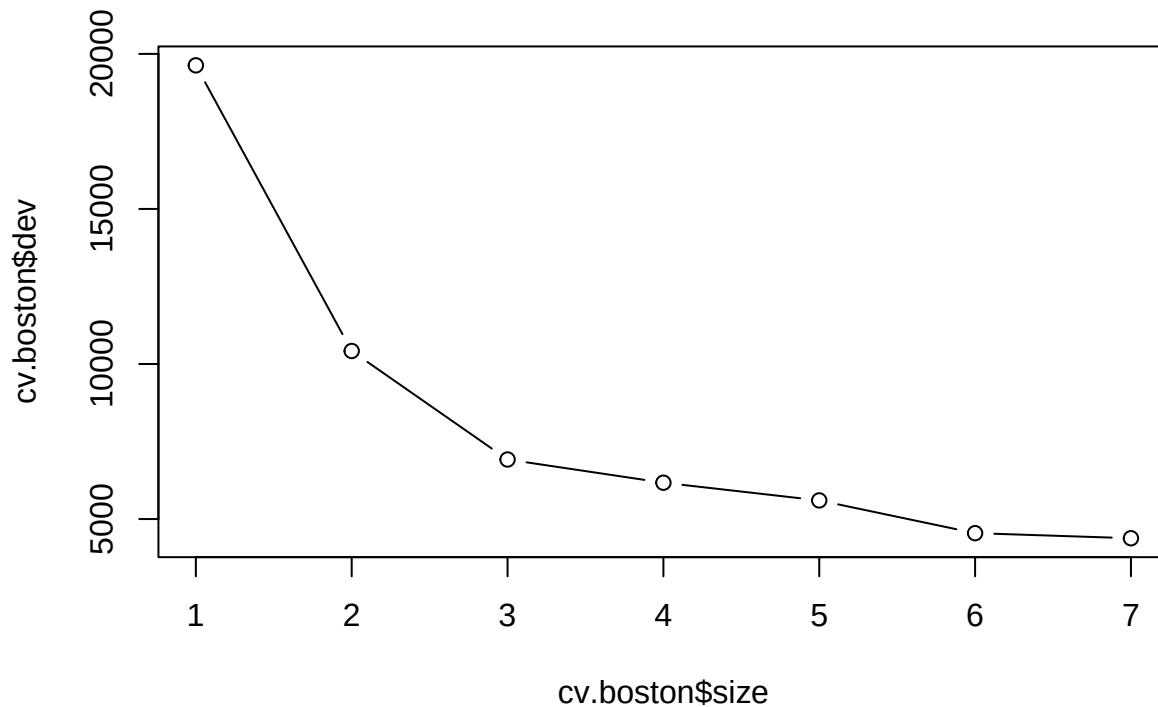
##           RMSE  Rsquared    MAE
## tree1  3.177645  0.8686412  2.41560
## reg1   4.254215  0.7645562  3.04222
```

Regression tree performs better than OLS regression model on the train data set.

It is worth noting that we could have fit a much bigger tree, by passing `control = tree.control(nobs = length(train), mindev = 0)` into the `tree()` function.

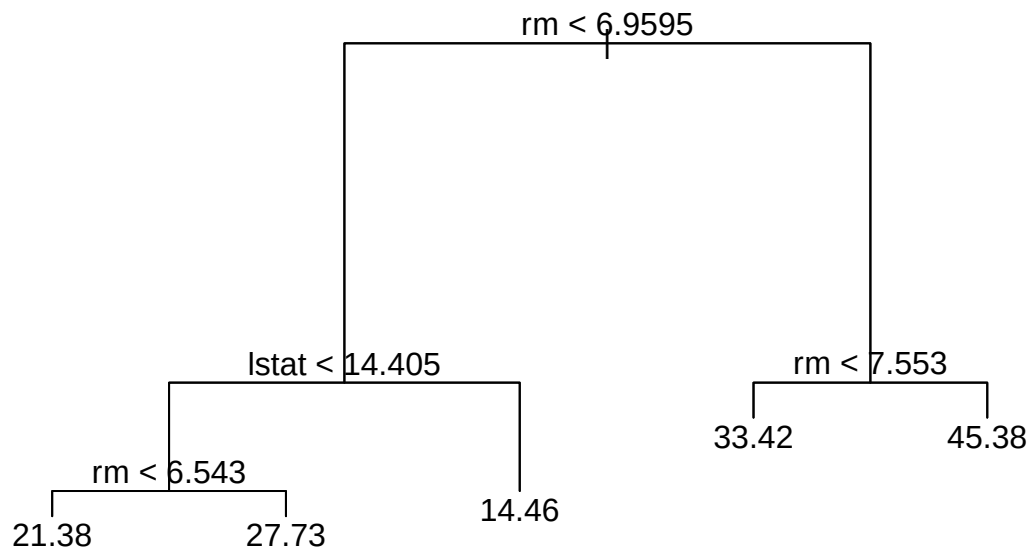
Now we use the `cv.tree()` function to see whether pruning the tree will improve performance.

```
cv.boston <- cv.tree(tree.boston)
plot(cv.boston$size, cv.boston$dev, type = "b")
```



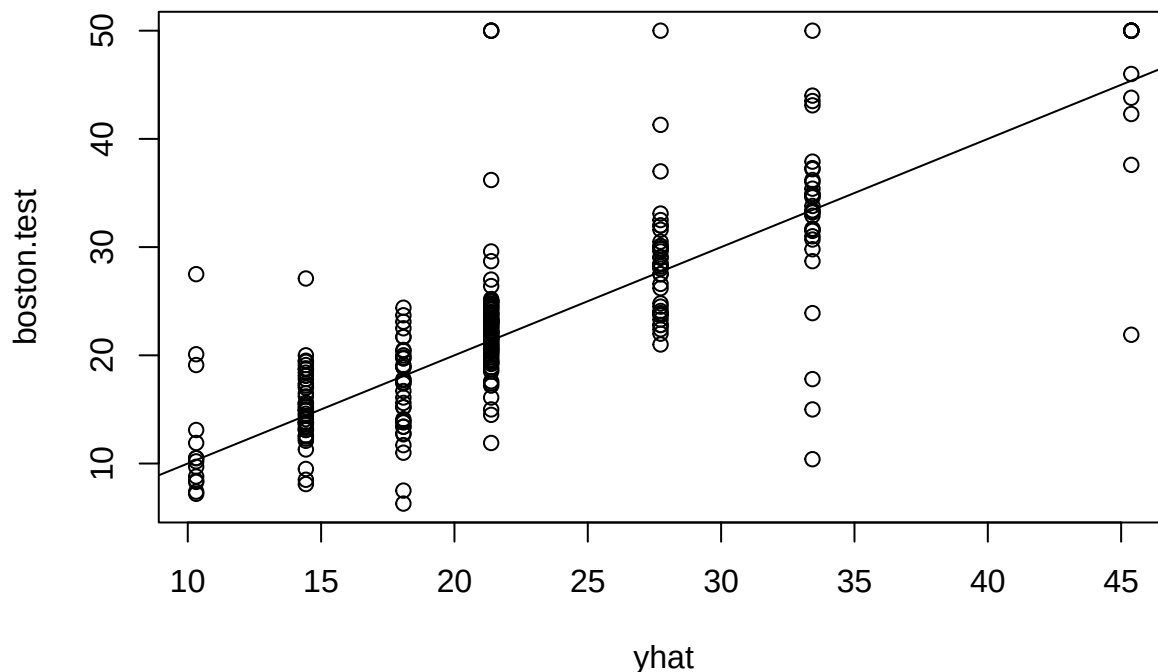
In this case, the most complex tree under consideration is selected by cross-validation. However, if we wish to prune the tree, we could do so as follows, using the `prune.tree()` function:

```
prune.boston <- prune.tree(tree.boston, best = 5)
plot(prune.boston)
text(prune.boston, pretty = 0)
```



In keeping with the cross-validation results, we use the unpruned tree to make predictions on the test set.

```
yhat <- predict(tree.boston, newdata = Boston[-train, ])
boston.test <- Boston[-train, "medv"]
plot(yhat, boston.test)
abline(0, 1)
```



```
mean((yhat - boston.test)^2)
```

```
## [1] 35.28688
```

```
#performance of the model on the Test data
```

```
#get the predicted values
```

```
PredictedTest<- predict(tree.boston, newdata = Boston[-train, ])
```

```
#create a data frame with 2 columns actual and predicted values
```

```
ModelTest<-data.frame(obs = Boston[-train, ]$medv, pred=PredictedTest)
```

```
#calculate performance statistics
```

```
tree1.test<-defaultSummary(ModelTest)
```

```
#lets compare this against regression model
```

```
PredictedTrain.reg.test<- predict(f, newdata = Boston[-train, ])
```

```
#create a data frame with 2 columns actual and predicted values
```

```
ModelTrain.reg.test<-data.frame(obs = Boston[-train, ]$medv, pred=PredictedTrain.reg.test)
```

```
#calculate performance statistics
```

```
reg1.test<-defaultSummary(ModelTrain.reg.test)
```

```
out.test<-rbind(tree1.test,reg1.test)
```

```
out.test
```

```
##           RMSE  Rsquared    MAE
## tree1.test 5.940276 0.6184571 3.692607
## reg1.test  5.258314 0.6964472 3.677885
```

Again, regression tree outperforms OLS regression on the test data set.

In other words, the test set MSE associated with the regression tree is 35.29. The square root of the MSE is therefore around 5.941, indicating that this model leads to test predictions that are (on average) within approximately \$5,941 of the true median home value for the census tract. ## Bagging and Random Forests

Here we apply bagging and random forests to the `Boston` data, using the `randomForest` package in R. The exact results obtained in this section may depend on the version of R and the version of the `randomForest` package installed on your computer. Recall that bagging is simply a special case of a random forest with

$m = p$. Therefore, the `randomForest()` function can be used to perform both random forests and bagging. We perform bagging as follows:

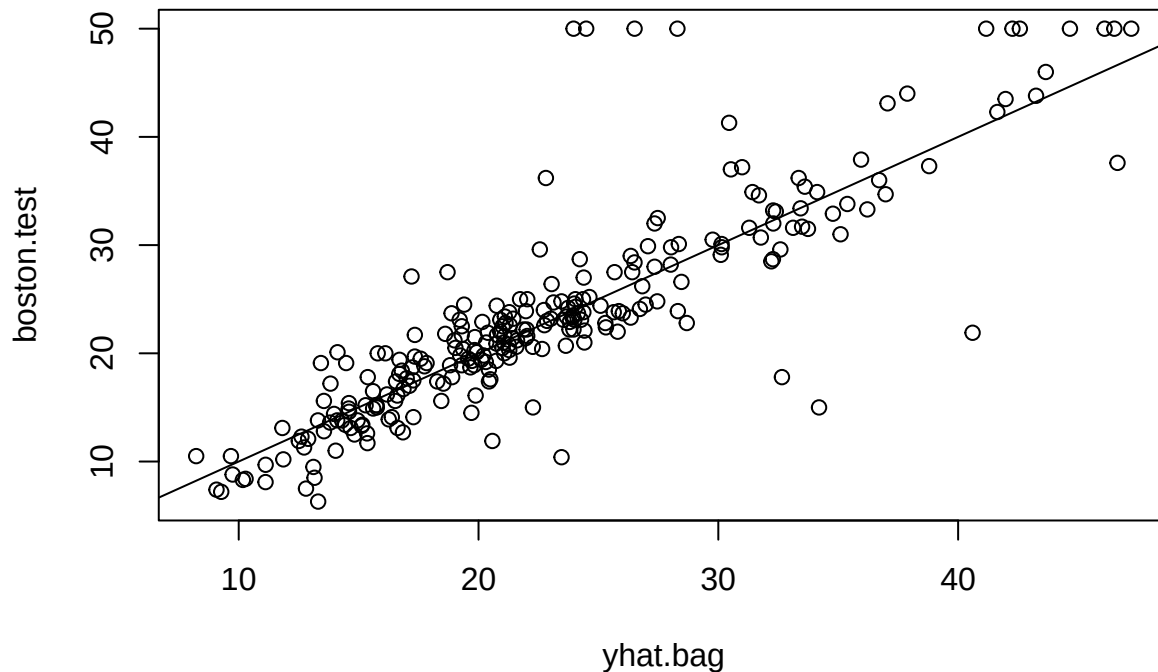
```
library(randomForest)

## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
## The following object is masked from 'package:ggplot2':
##
##      margin
Boston <- read.csv("/cloud/project/Data/Boston.csv")
set.seed(1)
bag.boston <- randomForest(medv ~ ., data = Boston,
  subset = train, mtry = 12, importance = TRUE)
bag.boston

##
## Call:
## randomForest(formula = medv ~ ., data = Boston, mtry = 12, importance = TRUE,      subset = train)
##              Type of random forest: regression
##              Number of trees: 500
## No. of variables tried at each split: 12
##
##              Mean of squared residuals: 10.92329
##              % Var explained: 85.79
```

The argument `mtry = 12` indicates that all 12 predictors should be considered for each split of the tree—in other words, that bagging should be done. How well does this bagged model perform on the test set?

```
yhat.bag <- predict(bag.boston, newdata = Boston[-train, ])
plot(yhat.bag, boston.test)
abline(0, 1)
```



```
mean((yhat.bag - boston.test)^2)
```

```
## [1] 22.70458
```

The test set MSE associated with the bagged regression tree is 23.42, about two-thirds of that obtained using an optimally-pruned single tree. We could change the number of trees grown by `randomForest()` using the `ntree` argument:

```
bag.boston <- randomForest(medv ~ ., data = Boston,
  subset = train, mtry = 12, ntree = 25)
yhat.bag <- predict(bag.boston, newdata = Boston[-train, ])
mean((yhat.bag - boston.test)^2)
```

```
## [1] 22.09346
```

Growing a random forest proceeds in exactly the same way, except that we use a smaller value of the `mtry` argument. By default, `randomForest()` uses $p/3$ variables when building a random forest of regression trees, and $\sqrt{p}$ variables when building a random forest of classification trees. Here we use `mtry = 6`.

```
set.seed(1)
rf.boston <- randomForest(medv ~ ., data = Boston,
  subset = train, mtry = 6, importance = TRUE)
yhat.rf <- predict(rf.boston, newdata = Boston[-train, ])
mean((yhat.rf - boston.test)^2)
```

```
## [1] 19.42736
```

The test set MSE is 20.07; this indicates that random forests yielded an improvement over bagging in this case.

Using the `importance()` function, we can view the importance of each variable.

```
importance(rf.boston)
```

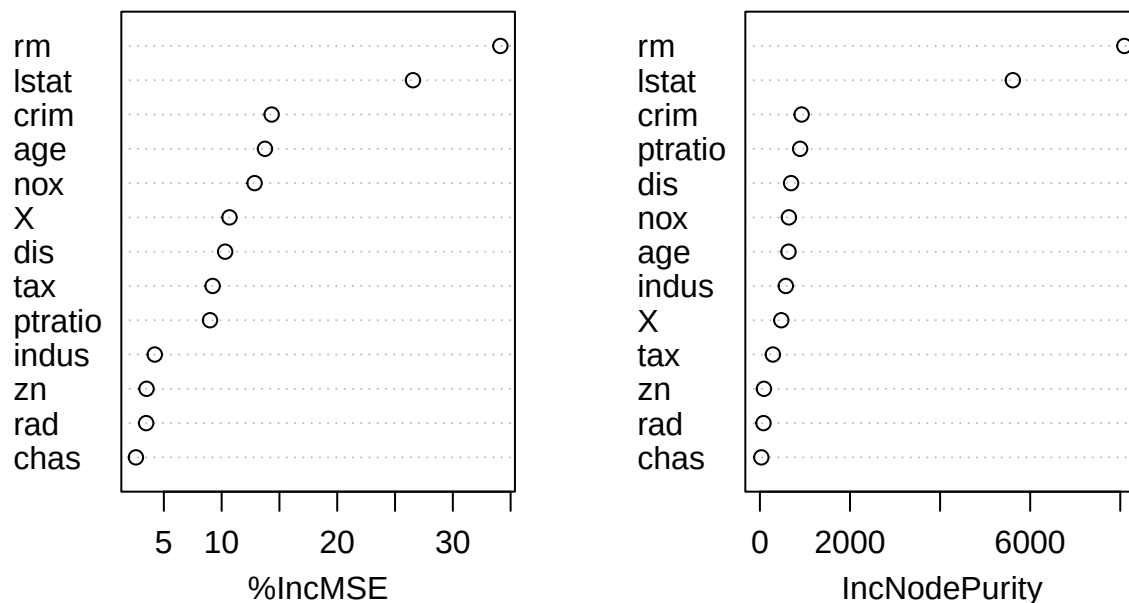
```
##           %IncMSE IncNodePurity
## X          10.681150      474.75329
## crim       14.325457      925.89669
```

## zn	3.503225	89.81559
## indus	4.214257	575.01010
## chas	2.583201	31.73100
## nox	12.852185	643.60377
## rm	34.111812	8090.77105
## age	13.748011	635.70110
## dis	10.304244	692.22136
## rad	3.469876	79.41280
## tax	9.220552	288.17302
## ptratio	8.995986	892.38364
## lstat	26.553001	5615.09645

Two measures of variable importance are reported. The first is based upon the mean decrease of accuracy in predictions on the out of bag samples when a given variable is permuted. The second is a measure of the total decrease in node impurity that results from splits over that variable, averaged over all trees (this was plotted in Figure 8.9). In the case of regression trees, the node impurity is measured by the training RSS, and for classification trees by the deviance. Plots of these importance measures can be produced using the `varImpPlot()` function.

```
varImpPlot(rf.boston)
```

rf.boston



The results indicate that across all of the trees considered in the random forest, the wealth of the community (`lstat`) and the house size (`rm`) are by far the two most important variables.

Boosting

Here we use the `gbm` package, and within it the `gbm()` function, to fit boosted regression trees to the `Boston` data set. We run `gbm()` with the option `distribution = "gaussian"` since this is a regression problem; if it were a binary classification problem, we would use `distribution = "bernoulli"`. The argument `n.trees = 5000` indicates that we want 5000 trees, and the option `interaction.depth = 4` limits the depth of each

tree.

```
library(gbm)
```

```
## Loaded gbm 2.2.2
```

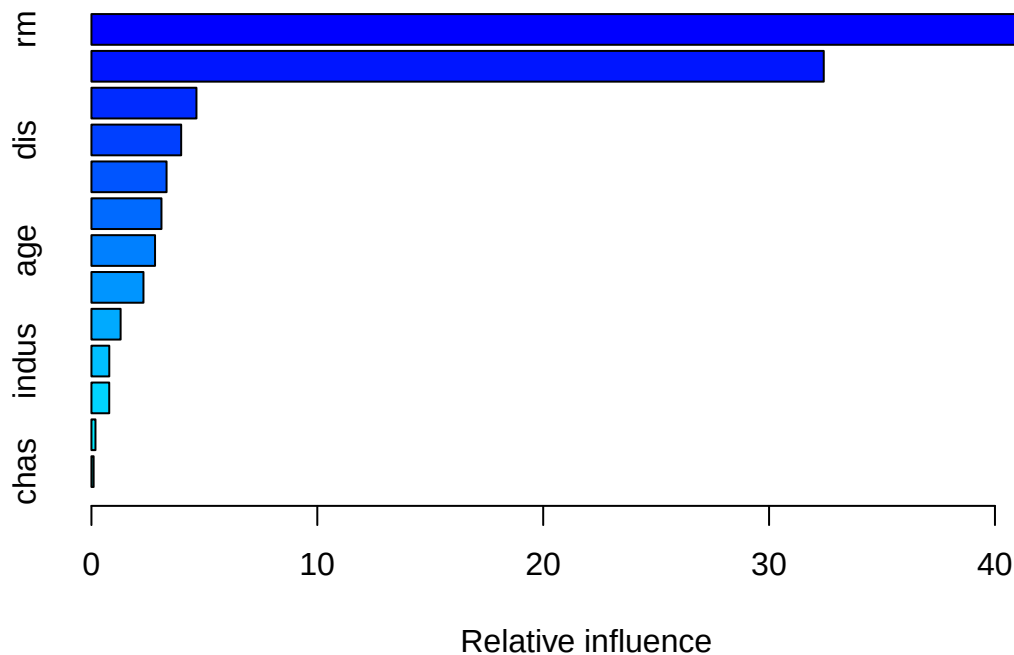
```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com
```

```
set.seed(1)
```

```
boost.boston <- gbm(medv ~ ., data = Boston[train, ], distribution = "gaussian", n.trees = 5000, interact
```

The `summary()` function produces a relative influence plot and also outputs the relative influence statistics.

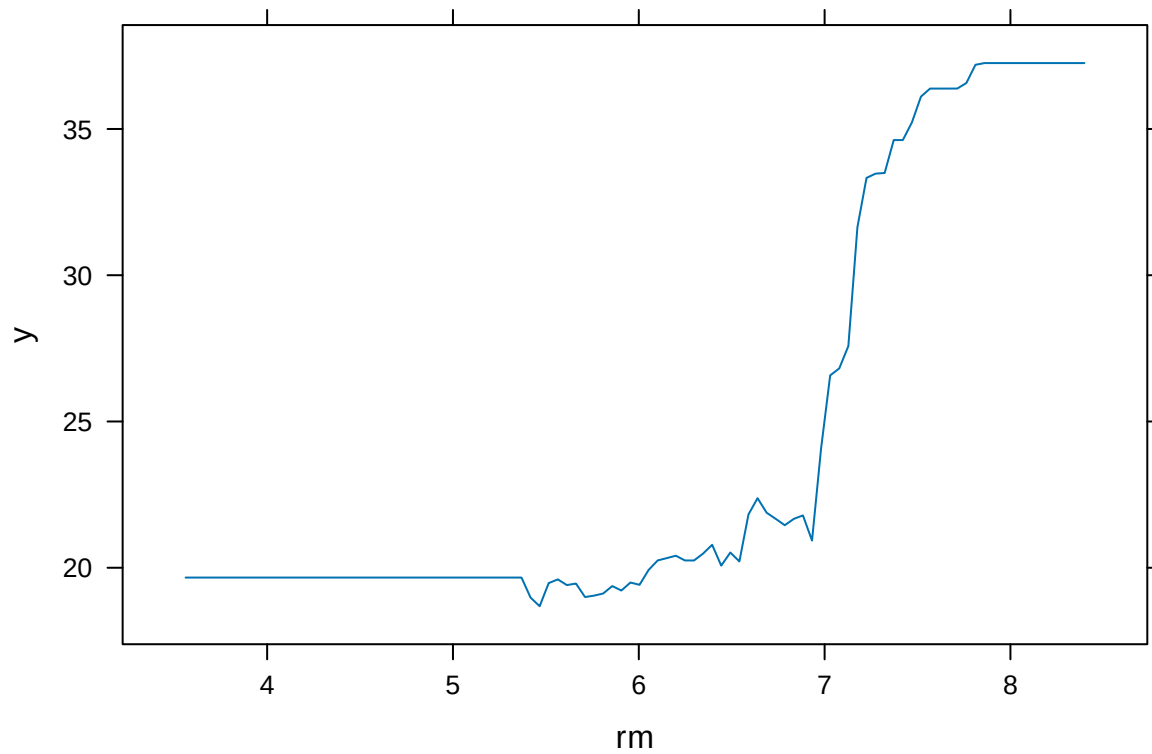
```
summary(boost.boston)
```



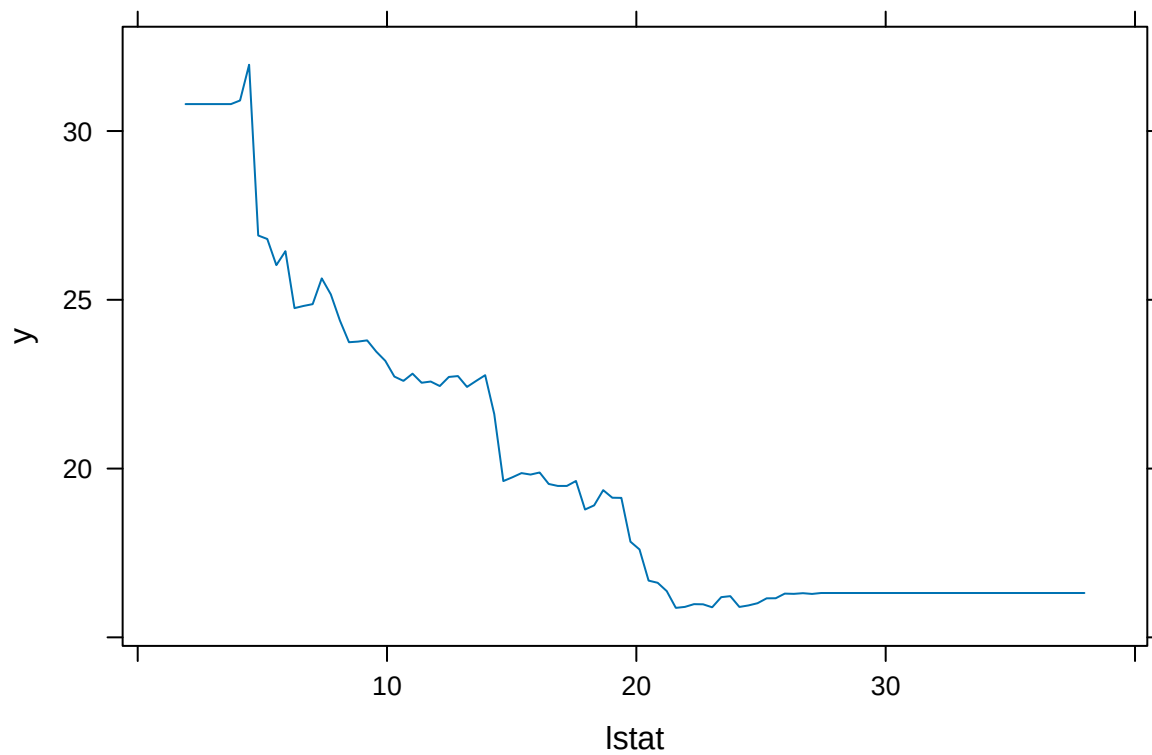
```
##      var      rel.inf
## rm      rm 44.26781902
## lstat    lstat 32.42064266
## crim     crim  4.64916159
## dis      dis  3.97318760
## nox      nox  3.32578804
## X        X   3.10060692
## age      age  2.81556564
## ptratio  ptratio 2.30353279
## tax      tax  1.29144997
## indus    indus 0.79000766
## rad      rad  0.78730818
## zn       zn   0.17828921
## chas     chas 0.09664071
```

We see that `lstat` and `rm` are by far the most important variables. We can also produce *partial dependence plots* for these two variables. These plots illustrate the marginal effect of the selected variables on the response after *integrating* out the other variables. In this case, as we might expect, median house prices are increasing with `rm` and decreasing with `lstat`.

```
plot(boost.boston, i = "rm")
```



```
plot(boost.boston, i = "lstat")
```



We now use the boosted model to predict `medv` on the test set:

```
yhat.boost <- predict(boost.boston,
  newdata = Boston[-train, ], n.trees = 5000)
mean((yhat.boost - boston.test)^2)
```

```
## [1] 17.43181
```

The test MSE obtained is 18.39: this is superior to the test MSE of random forests and bagging. If we want to, we can perform boosting with a different value of the shrinkage parameter λ in (8.10). The default value is 0.001, but this is easily modified. Here we take $\lambda = 0.2$.

```
boost.boston <- gbm(medv ~ ., data = Boston[train, ],
  distribution = "gaussian", n.trees = 5000,
  interaction.depth = 4, shrinkage = 0.2, verbose = F)
yhat.boost <- predict(boost.boston,
  newdata = Boston[-train, ], n.trees = 5000)
mean((yhat.boost - boston.test)^2)
```

```
## [1] 16.87695
```

In this case, using $\lambda = 0.2$ leads to a lower test MSE than $\lambda = 0.001$.

###using RPART Library

```
wine<- read.csv("/cloud/project/Data/whitewines.csv")
str(wine)
```

```
## 'data.frame':  4898 obs. of  12 variables:
## $ fixed.acidity      : num  7 6.3 8.1 7.2 7.2 8.1 6.2 7 6.3 8.1 ...
## $ volatile.acidity   : num  0.27 0.3 0.28 0.23 0.23 0.28 0.32 0.27 0.3 0.22 ...
## $ citric.acid        : num  0.36 0.34 0.4 0.32 0.32 0.4 0.16 0.36 0.34 0.43 ...
## $ residual.sugar     : num  20.7 1.6 6.9 8.5 8.5 6.9 7 20.7 1.6 1.5 ...
## $ chlorides          : num  0.045 0.049 0.05 0.058 0.058 0.05 0.045 0.045 0.049 0.044 ...
## $ free.sulfur.dioxide: num  45 14 30 47 47 30 30 45 14 28 ...
## $ total.sulfur.dioxide: num  170 132 97 186 186 97 136 170 132 129 ...
## $ density            : num  1.001 0.994 0.995 0.996 0.996 ...
## $ pH                 : num  3 3.3 3.26 3.19 3.19 3.26 3.18 3 3.3 3.22 ...
## $ sulphates          : num  0.45 0.49 0.44 0.4 0.4 0.44 0.47 0.45 0.49 0.45 ...
## $ alcohol            : num  8.8 9.5 10.1 9.9 9.9 10.1 9.6 8.8 9.5 11 ...
## $ quality            : int  6 6 6 6 6 6 6 6 6 6 ...
```

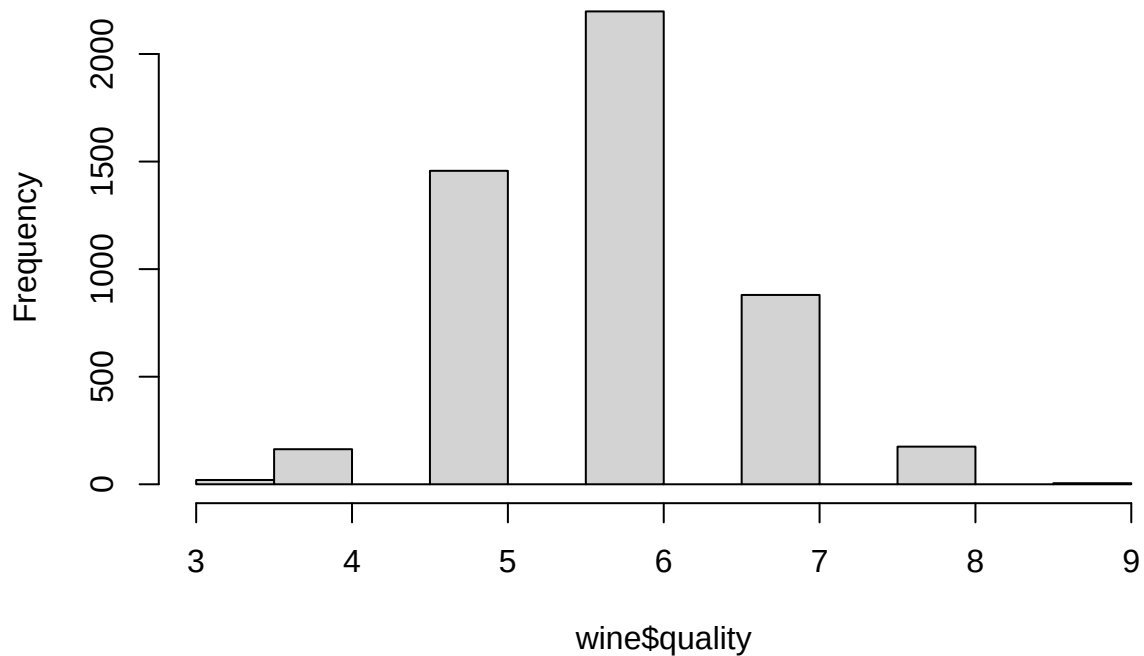
```
head(wine ,10)
```

```
##      fixed.acidity volatile.acidity citric.acid residual.sugar chlorides
## 1           7.0           0.27           0.36           20.7           0.045
## 2           6.3           0.30           0.34           1.6           0.049
## 3           8.1           0.28           0.40           6.9           0.050
## 4           7.2           0.23           0.32           8.5           0.058
## 5           7.2           0.23           0.32           8.5           0.058
## 6           8.1           0.28           0.40           6.9           0.050
## 7           6.2           0.32           0.16           7.0           0.045
## 8           7.0           0.27           0.36           20.7           0.045
## 9           6.3           0.30           0.34           1.6           0.049
## 10          8.1           0.22           0.43           1.5           0.044
##      free.sulfur.dioxide total.sulfur.dioxide density  pH sulphates alcohol
## 1              45              170 1.0010 3.00      0.45      8.8
## 2              14              132 0.9940 3.30      0.49      9.5
## 3              30              97 0.9951 3.26      0.44     10.1
```

```
## 4          47          186 0.9956 3.19      0.40      9.9
## 5          47          186 0.9956 3.19      0.40      9.9
## 6          30           97 0.9951 3.26      0.44     10.1
## 7          30          136 0.9949 3.18      0.47      9.6
## 8          45          170 1.0010 3.00      0.45      8.8
## 9          14          132 0.9940 3.30      0.49      9.5
## 10         28          129 0.9938 3.22      0.45     11.0
##   quality
## 1         6
## 2         6
## 3         6
## 4         6
## 5         6
## 6         6
## 7         6
## 8         6
## 9         6
## 10        6
```

```
hist(wine$quality)
```

Histogram of wine\$quality



The wine quality values appear to follow a fairly normal, bell-shaped distribution, centered around a value of six. This makes sense intuitively, because most wines are of average quality; few are particularly bad or good.

```
summary(wine)
```

```
## fixed.acidity  volatile.acidity  citric.acid    residual.sugar
## Min.   : 3.800    Min.   :0.0800  Min.   :0.0000  Min.   : 0.600
## 1st Qu.: 6.300    1st Qu.:0.2100  1st Qu.:0.2700  1st Qu.: 1.700
## Median : 6.800    Median :0.2600  Median :0.3200  Median : 5.200
## Mean   : 6.855    Mean   :0.2782  Mean   :0.3342  Mean   : 6.391
```

```
## 3rd Qu.: 7.300    3rd Qu.:0.3200    3rd Qu.:0.3900    3rd Qu.: 9.900
## Max.    :14.200    Max.    :1.1000    Max.    :1.6600    Max.    :65.800
## chlorides      free.sulfur.dioxide total.sulfur.dioxide    density
## Min.    :0.00900    Min.    : 2.00    Min.    : 9.0    Min.    :0.9871
## 1st Qu.:0.03600    1st Qu.: 23.00    1st Qu.:108.0    1st Qu.:0.9917
## Median :0.04300    Median : 34.00    Median :134.0    Median :0.9937
## Mean    :0.04577    Mean    : 35.31    Mean    :138.4    Mean    :0.9940
## 3rd Qu.:0.05000    3rd Qu.: 46.00    3rd Qu.:167.0    3rd Qu.:0.9961
## Max.    :0.034600    Max.    :289.00    Max.    :440.0    Max.    :1.0390
## pH            sulphates          alcohol          quality
## Min.    :2.720    Min.    :0.2200    Min.    : 8.00    Min.    :3.000
## 1st Qu.:3.090    1st Qu.:0.4100    1st Qu.: 9.50    1st Qu.:5.000
## Median :3.180    Median :0.4700    Median :10.40    Median :6.000
## Mean    :3.188    Mean    :0.4898    Mean    :10.51    Mean    :5.878
## 3rd Qu.:3.280    3rd Qu.:0.5500    3rd Qu.:11.40    3rd Qu.:6.000
## Max.    :3.820    Max.    :1.0800    Max.    :14.20    Max.    :9.000
```

There are outliers in the data. Even though trees are fairly robust to messy data, it is always prudent to check for severe problems. For now, we'll assume that the data is reliable.

lets do 70 and 30 split

```
set.seed(1023)
sample_ind<-sample(nrow(wine),round(0.7*nrow(wine)))
wine_train <- wine[sample_ind, ]
wine_test  <- wine[-sample_ind, ]
#install.packages("rpart")
library(rpart)
m.rpart <- rpart(quality ~ ., data = wine_train)
m.rpart
```

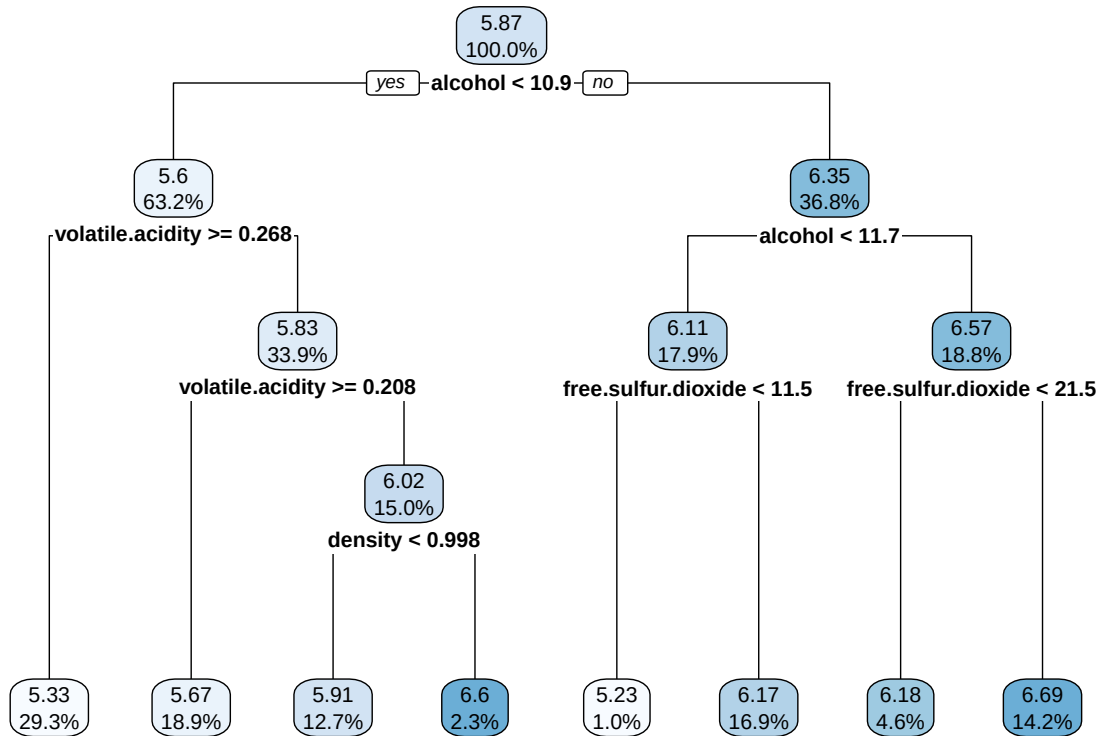
```
## n= 3429
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 3429 2641.81600 5.873141
##    2) alcohol< 10.85 2168 1249.07300 5.598247
##      4) volatile.acidity>=0.2675 1005 428.99900 5.332338 *
##      5) volatile.acidity< 0.2675 1163 687.60620 5.828031
##        10) volatile.acidity>=0.2075 648 324.29480 5.674383 *
##        11) volatile.acidity< 0.2075 515 328.76500 6.021359
##          22) density< 0.99788 435 241.85290 5.914943 *
##          23) density>=0.99788 80 55.20000 6.600000 *
##    3) alcohol>=10.85 1261 947.24980 6.345757
##      6) alcohol< 11.74167 615 444.03250 6.113821
##        12) free.sulfur.dioxide< 11.5 35 38.17143 5.228571 *
##        13) free.sulfur.dioxide>=11.5 580 376.77760 6.167241 *
##      7) alcohol>=11.74167 646 438.63780 6.566563
##        14) free.sulfur.dioxide< 21.5 159 111.06920 6.176101 *
##        15) free.sulfur.dioxide>=21.5 487 295.41270 6.694045 *
```

For each node in the tree, the number of examples reaching the decision point is listed. For instance, all 3,429 examples begin at the root node, of which 2,155 have alcohol < 10.85 and 1,274 have alcohol >= 10.85. Because alcohol was used first in the tree, it is the single most important predictor of wine quality.

Nodes indicated by * are terminal or leaf nodes, which means that they result in a prediction (listed here as

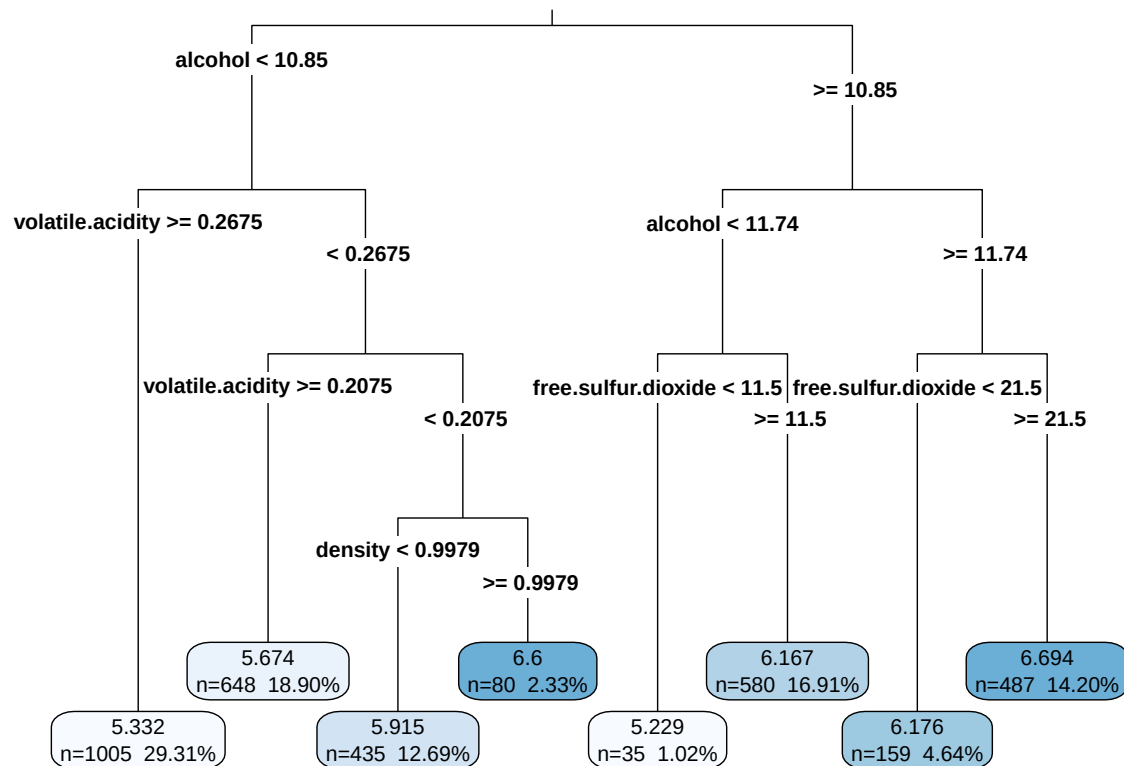
yval). For example, node 4 has a yval of 5.367. When the tree is used for predictions, any wine samples with $\text{alcohol} < 10.85$ and $\text{volatile.acidity} < 0.2525$ would therefore be predicted to have a quality value of 5.367.

```
library(rpart.plot)
rpart.plot(m.rpart, digits = 3)
```



The `fallen.leaves` parameter forces the leaf nodes to be aligned at the bottom of the plot, while the `type` and `extra` parameters affect the way the decisions and nodes are labeled. The numbers 3 and 101 refer to specific style formats, which can be found in the command's documentation, or via experimentation with various numbers.

```
rpart.plot(m.rpart, digits = 4, fallen.leaves = TRUE, type = 3, extra = 101)
```



#evaluating model performance

```
p.rpart <- predict(m.rpart, wine_test)
summary(p.rpart)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      5.229  5.332   5.915   5.872  6.167   6.694
```

```
summary(wine_test$quality)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      3.000  5.000   6.000   5.889  6.000   9.000
```

```
cor(p.rpart, wine_test$quality)
```

```
## [1] 0.5185679
```

```
PredictedTest1<- predict(m.rpart, wine_test)
```

#create a data frame with 2 columns actual and predicted values

```
ModelTest1<-data.frame(obs = wine_test$quality, pred=PredictedTest1)
```

#calculate performance statistics

```
defaultSummary(ModelTest1)
```

```
##      RMSE Rsquared    MAE
## 0.7726936 0.2689126 0.6115625
```

A correlation of 0.53 is certainly acceptable. However, the correlation only measures how strongly the predictions are related to the true value; it is not a measure of how far off the predictions were from the true values.

```
library(caret)
```

```
PredictedTest<-predict(m.rpart, wine_test)
```

```
ModelTest<-data.frame(obs = wine_test$quality, pred=PredictedTest)
```

```
defaultSummary(ModelTest)
```

```
##      RMSE  Rsquared      MAE
## 0.7726936 0.2689126 0.6115625
```

##Pruning

cp: complexity parameter. Any split that does not decrease the overall lack of fit by a factor of cp is not attempted. For instance, with anova splitting, this means that the overall R-squared must increase by cp at each step. The main role of this parameter is to save computing time by pruning off splits that are obviously not worthwhile. Essentially, the user informs the program that any split which does not improve the fit by cp will likely be pruned off by cross-validation, and that hence the program need not pursue it.

```
library(faraway)
```

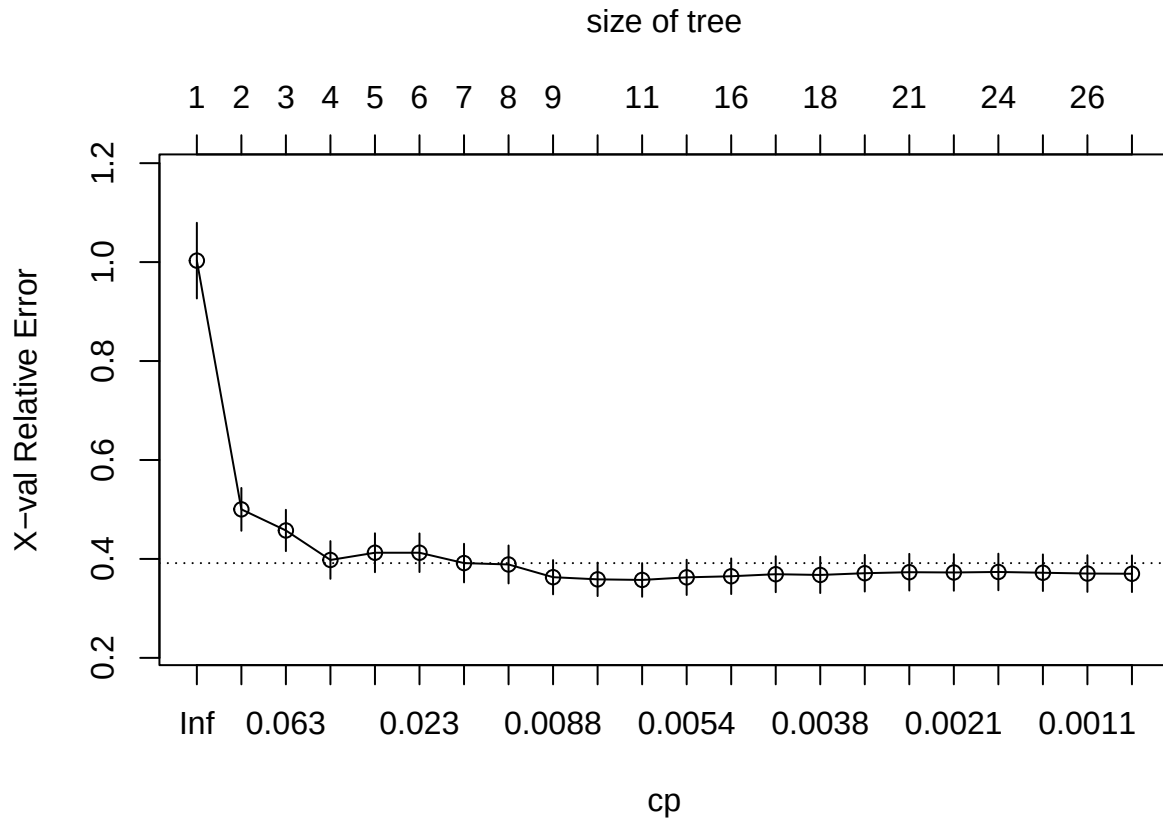
```
##
## Attaching package: 'faraway'
## The following object is masked from 'package:rpart':
##
##      solder
## The following object is masked from 'package:lattice':
##
##      melanoma
set.seed(123)
tmod<-rpart(O3~., ozone, cp=0.001)
printcp(tmod)
```

```
##
## Regression tree:
## rpart(formula = O3 ~ ., data = ozone, cp = 0.001)
##
## Variables actually used in tree construction:
## [1] doy      dpg      humidity ibh      ibt      temp      vh      vis
##
## Root node error: 21115/330 = 63.986
##
## n= 330
##
##      CP nsplit rel error  xerror    xstd
## 1  0.5456993      0  1.00000 1.00296 0.076477
## 2  0.0736591      1  0.45430 0.50019 0.043274
## 3  0.0535415      2  0.38064 0.45752 0.041780
## 4  0.0267557      3  0.32710 0.39790 0.038133
## 5  0.0232760      4  0.30034 0.41249 0.039340
## 6  0.0231021      5  0.27707 0.41252 0.039022
## 7  0.0153249      6  0.25397 0.39159 0.039024
## 8  0.0109137      7  0.23864 0.38880 0.038336
## 9  0.0070746      8  0.22773 0.36305 0.034622
## 10 0.0059918      9  0.22065 0.35872 0.033802
## 11 0.0059317     10  0.21466 0.35754 0.034008
## 12 0.0049709     12  0.20280 0.36274 0.035600
## 13 0.0047996     15  0.18789 0.36498 0.036045
## 14 0.0044712     16  0.18309 0.36910 0.036391
## 15 0.0031921     17  0.17861 0.36760 0.036586
## 16 0.0022152     19  0.17223 0.37110 0.037023
```



```
## 17 0.0020733    20  0.17002 0.37314 0.037073
## 18 0.0020297    22  0.16587 0.37261 0.036929
## 19 0.0014432    23  0.16384 0.37363 0.036966
## 20 0.0011322    24  0.16240 0.37209 0.037019
## 21 0.0011035    25  0.16126 0.37043 0.037059
## 22 0.0010000    26  0.16016 0.37004 0.037069
```

```
plotcp(tmod)
```



In this table, we see the value of the cp parameter, the number of splits in the tree, the RSS of the tree divided by the RSS of the null tree.

xerror denotes the cross-validated error which is also scaled by the RSS of the null tree. Since the partition of the data into 10 parts is random, this CV error is also random, which makes the given standard error useful. The random division also means that if you repeat this command, you will not get the same answer unless you use `set.seed`. We can select the size of the tree by minimizing the value of xerror and selecting the corresponding value of CP.

We can select the smallest tree with a CV error within one standard error of the minimum — in this case, $0.350 + 0.035 = 0.385$. So we would take the seven-split tree.

```
library(rpart.plot)
rpart.plot(tmod, type=3)
```

